

Proof-of-Instant-Settlement:

Cryptographically Binding PSP-Attested Instant Payments to Confidential On-Chain State

Juliano Pereira Sales
Harpo | harpo-zk
tzdesing@gmail.com

Marco Túlio Rocha Nascimento
Harpo | harpo-zk
marcotulio.nascimento@gmail.com

September 7, 2026

Abstract

On-chain confidentiality layers (Zeto, Railgun, Rayls/Enygma) prove *confidential transfers* between on-chain accounts, but, to the best of our knowledge, none of the public-EVM confidential-transfer systems considered here binds its private state to a *real fiat settlement*. For tokenized-asset settlement integrated with instant-payment rails — concretely Brazil’s central-bank instant-payment system — the question that matters is not “did an on-chain balance move” but “*did the fiat money actually settle, for this exact operation*”, and to answer it *without revealing* amount or counterparties. We present Proof-of-Instant-Settlement, a construction that cryptographically binds a specific instant payment, identified by its rail-level *EndToEndId* (E2EID) and *attested by a regulated PSP*, to an on-chain commitment via a Groth16 circuit, together with an attestation oracle that lets the chain verify — through an EIP-712 signature from a registered payment service provider (PSP) — that the E2EID corresponds to a payment the PSP reports as settled and credited. We are precise about what is proved and by whom: the construction proves a PSP-attested claim about the rail, not the rail’s own cryptographic truth; §8 states this trust boundary as the central fact about the system, not a footnote. The construction is *two-phase*: a commitment c_0 to the agreed terms is registered when the charge is created, before any payment exists, and the settlement commitment c is proven to carry the same amount and charge identifier — which gives an asset escrow something to lock onto: the commitment/escrow primitive a delivery-versus-payment composition needs, though composing it with the payment leg into a single atomic transition is not itself demonstrated here (§11). The PSP signs over c_0 rather than over the amount, so the protocol verifies *value consistency without the value ever appearing on-chain*. We give the circuit (1,251 R1CS constraints), the on-chain verification cost measured on a live public testnet, the gas of the complete settlement and attestation calls, and an explicit, honest threat model. We are deliberate about the residual trust: the oracle replaces “trust the submitter” with “trust a PSP whose key is registered, whose attestations expire, and whose every attestation is auditable on-chain” — it does *not* remove trust in the PSP. Removing that last assumption (via a central-bank attestation, dual attestation over an open-banking API, or zkTLS over the PSP endpoint) is stated as the core open problem. We additionally report a well-determinedness constraint analysis of the settlement circuits, a family of *confidential compliance predicates* bound to a PSP-attested counterparty commitment, and a post-quantum audit-disclosure channel (ML-KEM, FIPS-203). This is a research prototype: validated end-to-end on the XDC Apothem testnet, not yet run in production, with the reference implementation currently maintained by a single maintainer.

What this construction proves, stated as a boundary. *Cryptographically:* the same amount and txid underlie c_0 and c (Property 3); the settlement tag is deterministic in `e2eld` under a registered, immutable domain key and single-use within that domain (Property 6); the counterparty commitment consistent with the settlement proof is the one the PSP attested, or the operation does not close (Properties 4 and 7). *Institutionally:* a registered PSP attested that it observed this payment settle and credit, over this c_0 and this counterparty commitment (Assumption (I1)). *Not proved:* that the central bank’s own infrastructure settled the payment (§8 discusses four candidate paths, none evaluated here); that the attesting PSP is honest, only that a registered key signed (Property 7); economic irreversibility of the settlement (§4.2); unlinkability across operations (§8, “confidentiality is not unlinkability”); or soundness under a production-grade trusted setup (§8 states the reference artifacts’ setup as development-grade).

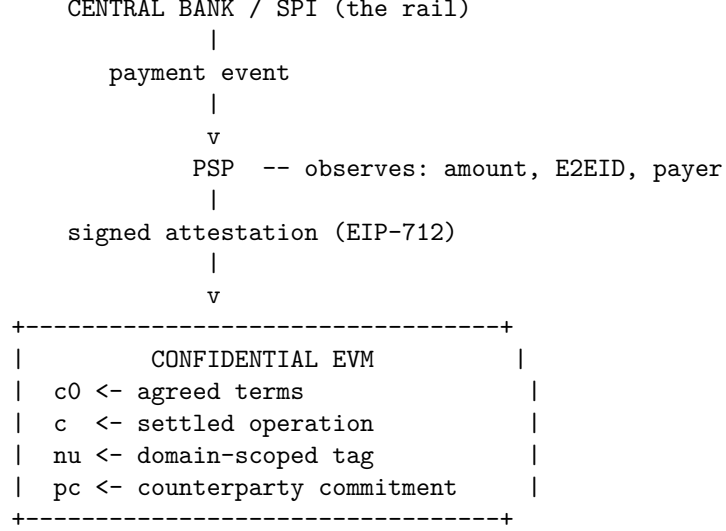
1 Introduction

Confidential-transfer systems on EVM chains prove, in zero knowledge, that a private balance update is well-formed (values conserved, ownership valid, no double-spend). This is by now a commodity: Zeto, Railgun, and the Brazilian central-bank-adjacent Rayls/Enygma all rely on the same primitive family (Groth16 or PLONK over BN254/BLS12-381, Pedersen/Poseidon commitments, UTXO or account models with nullifiers) [3, 4]. What none of them addresses is the *link to a real payment rail*: the on-chain proof says nothing about whether fiat currency actually moved.

Scope, stated precisely. Proving that a fiat payment happened in order to release an on-chain escrow is *not* new. ZKP2P [5] has done exactly that in production since 2023, using zk-email and zkTLS over consumer payment rails, and is the honest prior art for the general idea. Canton [6] is the incumbent for institutional delivery-versus-payment with privacy, on a permissioned substrate. Our claim is narrower and should be read that way: *within the confidential-transfer family on public EVM chains*, we bind the confidential on-chain state to a specific instant payment as attested by a regulated PSP — not to the central bank’s own settlement record, which the construction never touches (§8) — and do so while also answering regulatory predicates confidentially. Relative to ZKP2P, the difference is the institutional setting (a regulated PSP attestation rather than a consumer TLS/email proof) and the confidential-compliance model; relative to Canton and to Brazil’s Drex pilots, the difference is that this runs on a public EVM chain with no permissioned substrate.

For a settlement use case — e.g. delivery-versus-payment for a tokenized credit operation — the binding to fiat is the whole point. The paying institution must demonstrate that *a specific instant payment* was made, tied to *this* on-chain operation, while keeping the amount, the parties, and the payment identifiers confidential.

What is actually proved, stated as the thesis. The construction does not cryptographically reach the central bank; it reaches a regulated PSP that observed the rail. Stating that plainly, up front, is more useful than leaving a reader to infer it from the threat model in §8. The sketch below is the trust chain in miniature; Figure 1, after the contribution list, gives the full protocol flow it summarizes:



Three different things get called “proof” in this design, and conflating them is the single easiest way to overstate what it delivers. *Level 1, the ZK proof*: knowledge of (amount, salt₀, salt, e2eld, txid, k, party, salt_p) consistent with the public commitments (§3) — this is unconditional, under (A1). *Level 2, the PSP attestation*: that a specific registered key signed a specific statement (§4, Property 7) — authentication, not truthfulness. *Level 3, rail truth*: that the central-bank infrastructure itself attests the payment — this is what the construction does *not* provide today; §8 discusses four candidate paths toward it, none evaluated here. What we deliver is Level 1 + Level 2. The gap between that and Level 3 is the residual Trusted-PSP assumption (Assumption (I1)), stated as the central fact about this system, not a footnote to it.

The general pattern, Pix as its instantiation. Nothing in §3–§4 is specific to Pix beyond the canonicalization profile of Appendix A: an *external settlement binding primitive* takes (a) a rail-level identifier born only at settlement (an E2EID, but equally a FedNow trace number or a SEPA Instant end-to-end reference), (b) an attestor able to evaluate one hash over what it observed (any ISO 20022 PSP, not only a Pix one), and (c) a pre-existing commitment to bind it to. Proof-of-Instant-Settlement is that primitive instantiated for Pix; we keep the one name for the concrete system throughout, rather than introducing an acronym for the general pattern, but the generality is the point of stating it here rather than leaving a reader to infer it from the canonicalization appendix.

Contribution.

1. A two-phase commitment scheme and a Groth16 circuit `settlement_verify_v2` that binds the agreed terms (amount, txid), registered *before* the payment exists, to the settled payment (e2eld), exposing a keyed settlement tag ν and the on-chain anchor K of the key that produced it (§3).
2. An attestation oracle whose receipt is signed over the terms commitment c_0 rather than over the value — a mechanism we call *commitment-carried value consistency*: the attestor independently recomputes the commitment from the amount it actually observed settling, so the chain verifies value consistency while no value or payment identifier ever appears on-chain. The mechanism is not specific to this rail: any ISO 20022 attestor that can evaluate one hash gains the same property. The receipt carries an expiry, and the oracle exposes an explicit reversal path for the rail’s refund window (§4).

3. Three *confidential compliance predicates* — value-within-limit, non-membership in an attested sanctions set, and membership in an attested KYC set — bound to a *PSP-attested* counterparty commitment, so the counterparty they speak about is the one that actually paid (§5).
4. A crypto-agile, on-chain-governed *post-quantum audit channel* (ML-KEM-768, FIPS-203) and a preliminary in-circuit binding of the disclosure ciphertext to the commitment (§6).
5. A concrete, *measured* implementation report: circuit size at a stated optimization level, proving time, and the gas of the *complete* calls, not of the verifier in isolation (§9).
6. An explicit threat model that states exactly what is and is *not* protected, including liveness, key compromise, and the residual trust in the PSP (§8).

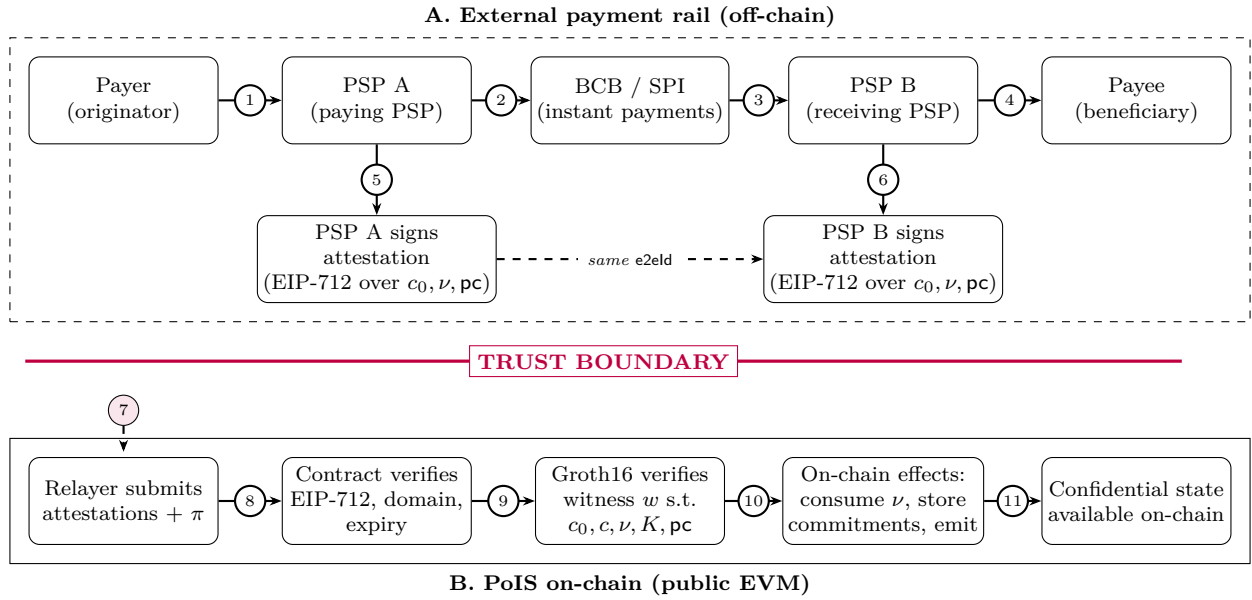


Figure 1: PoIS protocol flow. **A (off-chain)**: (1) the payer initiates a Pix payment; (2)–(4) the rail carries it through both PSPs to the payee; (5, 6) each PSP that needs to sign attests what it observed, over the commitments — never the plaintext value or identifiers. **Trust boundary**: everything above is off-chain and PSP-attested; everything below is on-chain and cryptographically checked — the off-chain payment event reaches the on-chain state only through the PSP attestation and the ZK consistency checks of §3–§4. **B (on-chain)**: (7, 8) a relay submits the attestation(s) and the Groth16 proof π ; (9) the contract checks the EIP-712 signature, domain anchor and expiry; (10) the proof is verified; (11) the tag is consumed and confidential commitments are recorded. This is a schematic of the flow described in §3–§4; the reference implementation’s negative cases (§9) exercise each check in box B individually.

2 Preliminaries

Instant payment / E2EID. The `EndToEndId` is the unique identifier that accompanies an instant-payment transaction end to end. It is worth being precise about its provenance, because the design depends on it: the E2EID is generated *by the payer’s PSP at initiation* — the institution code

(ISPB) embedded in it is the *payer's* — and it exists for transactions that are subsequently *rejected* as well as for those that settle. The central-bank infrastructure (SPI) validates its uniqueness; it does not mint it. The consequence is the premise of this work: *possession of an E2EID does not imply settlement*. An identifier alone is a claim, not a fact, which is precisely why the attestation oracle of §4 exists.

Field encoding and canonicalization. Strings (the E2EID, the PSP charge id `txid`) are mapped to elements of the BN254 scalar field $\mathbb{F}_p$ off-chain via $\text{stringToField}(s) = \text{keccak256}(s) \bmod p$. The hash is taken over the *exact bytes returned by the PSP API*: UTF-8, with no case-folding, no trimming, and no re-serialization. This is not a detail — the PSP and the institution must hash byte-identical inputs or the attestation and the proof will refer to different field elements and the settlement will simply fail to close. Implementations must therefore treat the API response bytes, not a parsed-and-reprinted string, as the canonical input.

The counterparty identifier committed in `pc` (§3) needs its own canonicalization rule, $\text{CanonicalID}_{\text{rail}}$, and that rule is necessarily *per-rail*: neither the on-chain contracts nor the circuits parse an identifier format — to them `pc` is an opaque field element, and any national ID / instant-payment rail (a US EIN over FedNow, an IBAN over SEPA Instant, and so on) plugs in by fixing its own $\text{CanonicalID}_{\text{rail}}$ and computing `pc` the same way on both sides. The PSP and the institution must apply the identical rule: two formattings of the same identifier are two different field elements, and `pc` computed by one party will not match `pc` committed by the other. Appendix A gives $\text{CanonicalID}_{\text{Pix}}$, the concrete rule this prototype and its pilot deployment use, kept out of the core protocol description deliberately: the construction is rail-agnostic, and a rule specific to Brazilian identifier formats has no place fixing what the core protocol requires elsewhere. Amounts, finally, are committed as the integer number of the settlement currency's smallest unit, parsed exactly from the API's decimal string — no rounding and no locale formatting — and the PSP recomputing c_0 must use that same integer: a value-consistency check that compares two encodings of the same amount fails closed, which is safe but useless.

Poseidon. A ZK-friendly hash over $\mathbb{F}_p$; we use the circomlib instantiations of arity 1 to 4 [2].

Groth16. A pairing-based zk-SNARK with constant-size proofs (8 field elements over BN254, ≈ 256 bytes) and cheap on-chain verification via the EVM `alt_bn128` precompiles (EIP-196/197) [1]. Groth16 requires a per-circuit trusted setup; see §8 for our stance on it.

3 Construction

3.1 Two-phase commitment

A single commitment $c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid})$, registered once at settlement, would be the simplest design, but it is only computable *after* the payment, because the E2EID is born there. The consequence is structural: at the moment the parties agree on terms there would be nothing on-chain that an asset escrow can lock onto, and the asset leg therefore could not be conditioned on the payment leg — which is exactly what a delivery-versus-payment composition needs. We therefore split the commitment in two.

At **charge creation**, before any payment exists, the orchestrator registers

$$c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}), \tag{1}$$

the commitment to the *agreed terms*. The escrow locks on c_0 . At **settlement**, once the payment has happened, the orchestrator registers

$$c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}). \quad (2)$$

The circuit proves that the *same* amount and the *same* `txid` appear in both, which is what ties the settled payment to the terms that were agreed before it existed. Nothing in the clear goes on-chain at either phase. The blinding factors salt_0 and salt_p are shared with the PSP at charge creation — and only these two, never salt — so that the PSP can recompute c_0 from the amount it actually settled, and pc from the debtor identifier in the payment message (§4).

3.2 Circuit `settlement_verify_v2`

- **Public signals, in this fixed order:** c_0 , c , the settlement tag ν , the domain-key anchor K , and the counterparty commitment pc — i.e. `pubSignals[0..4]`. `SettlementV2.finalize` indexes this array positionally (§3); the order is fixed by the circuit’s public-signal declaration and cross-checked manually against the contract’s indexing, since `verifyProof` itself only checks that the proof is valid *for the signals as given* — it enforces nothing about what a given position is supposed to mean, which is a property the calling contract, not the verifier, is responsible for getting right.
- **Private signals:** `amount`, salt_0 , salt , `e2eld`, `txid`, the domain key k , the counterparty *party*, and salt_p .
- **Constraints:**

$$c_0 \stackrel{!}{=} \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}) \quad (3)$$

$$c \stackrel{!}{=} \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}) \quad (4)$$

$$\nu \stackrel{!}{=} \text{Poseidon}(\text{e2eld}, k) \quad (5)$$

$$K \stackrel{!}{=} \text{Poseidon}(k) \quad (6)$$

$$\text{pc} \stackrel{!}{=} \text{Poseidon}(\text{party}, \text{salt}_p) \quad (7)$$

Equations (3) and (4) share the private witness *amount* and *txid*, which is the two-phase binding. Binding `txid` into both commitments prevents a valid proof for one $(\text{amount}, \text{e2eld})$ from being reused for a different charge.

Settlement tag: one symbol, keyed. We use a single symbol ν throughout for the settlement tag — in ZK terminology, ν is a *domain-scoped nullifier*: deterministic in `e2eld` within a fixed domain key k , and recorded as spent on first acceptance, but (as Property 6, §7, states precisely) not a nullifier over a global namespace. We keep “tag” as the primary term in the prose that follows because it is what the reference implementation names it, and flag the correspondence here once rather than throughout. The unkeyed variant `Poseidon(e2eld)` is not part of the construction (it is retained only as an ablation row in §9, to make the keying’s cost and necessity legible). The tag, together with the on-chain registry that records each accepted value as spent, enforces *single use*: a settled payment closes at most one operation. ν itself must be *deterministic* in `e2eld` — otherwise the same payment could be replayed under a fresh tag — yet not leak `e2eld`. The naïve `Poseidon(e2eld)` meets determinism but not hiding: the E2EID is structured and low-entropy,

$E\langle ISPB:8 \rangle \langle timestamp:minute \rangle \langle random \rangle$. The cheap attack is not brute force over the random suffix; it is *linkage by a party that already holds the E2EID legitimately*. The payer’s own bank, which is not a party to the on-chain operation, knows the E2EIDs it issued and can simply hash them and look for a match. We therefore key the tag: $\nu = \text{Poseidon}(e2eld, k)$ with k a domain secret carrying ≈ 254 bits (finding RT-5 of our own red team).

Anchoring the domain key. A keyed tag raises the question of who holds k . If k lives only with the participants and the PSP, the PSP becomes its custodian and uniqueness holds only under the informal premise that everyone in a domain used the same key. We therefore register $K = \text{Poseidon}(k)$ on-chain at domain setup and expose it as a public signal of the circuit. Two tags accepted under the same K provably come from the same k , so *intra-domain* uniqueness holds by construction rather than by convention. Cross-domain uniqueness is a different claim and rests on the PSP’s $e2eld \rightarrow txid$ bijection (Assumption (I1)).

ν is a selective-linkability primitive, not a privacy primitive against the key holder. Whoever holds k — the PSP, in this deployment — can compute $\text{Poseidon}(e2eld, k)$ for any $e2eld$ it observes and therefore map every ν in the domain back to the E2EID that produced it: the entire domain’s on-chain tag history is linkable, by construction, to whoever holds k . This is not an oversight to be fixed; it is what makes k do useful work at all, and it is the same holder as the one already trusted for attestation (Assumption (I1)) — keying ν adds no new party who can deanonymize the domain. What the key achieves is narrower and precisely stated: *public unlinkability for any observer without k* , against the concrete attack of §3 (a party that already holds E2EIDs legitimately hashing them to find a match) — combined with *intentional, domain-scoped linkability for whoever holds k* . Calling this “privacy” without the second half would overstate it.

Key rotation. The anchor makes K immutable for the lifetime of a domain, and this is deliberate: if a domain rotated k to k' , the same $e2eld$ would produce a fresh tag $\nu' = \text{Poseidon}(e2eld, k')$, and the tag registry — which by design cannot see $e2eld$ — would accept a second settlement of an already-settled payment. Rotation is therefore expressed as starting a *new* domain (new K , new escrows), and uniqueness across the two domains is not enforced by the circuit: like cross-domain uniqueness in general, it rests on the PSP’s $e2eld \rightarrow txid$ bijection (Assumption (I1)). A deployment that anticipates key compromise should treat this as an operational cost of the design, not a footnote. Put precisely: *domain separation is a security boundary*, the thing uniqueness (Property 6) is stated relative to, not merely a key-management convenience — crossing it is exactly where the unconditional guarantee ends and Assumption (I1) begins.

Counterparty commitment. The compliance predicates of §5 speak about a counterparty. For them to mean anything, that counterparty must be the one that actually paid. We therefore make $pc = \text{Poseidon}(\text{party}, \text{salt}_p)$ a public signal of the settlement circuit **and** a signed field of the PSP receipt (§4): the PSP computes pc over the debtor identifier (CPF/CNPJ) exactly as it appears in the payment message, canonicalized per §3 above, using the salt_p shared at charge creation, and **finalize** accepts only if the attested pc equals the proof’s public signal. For charges that do not pin a debtor at creation, the PSP computes pc at attestation time, when the payment message names the payer. Without both halves the predicates are decorative — the prover picks the party; with them, the counterparty the predicates speak about is, by construction, the counterparty that paid. We return to this in §5.

3.3 Protocol

1. The receiving institution creates a *real* instant-payment charge through a PSP (e.g. a PSP sandbox), obtaining a `txid`. The commitment c_0 to the agreed terms is registered on-chain and the asset escrow locks on it.
2. The charge (the instant-payment payload string and `txid`) is exchanged *encrypted, peer-to-peer and off-chain* between the two institutions, with an optional on-chain anchor of its hash. It is deliberately *not* written encrypted to the chain: a ciphertext over a payment key, a value and a charge identifier, recorded permanently on a public ledger, is a harvest-now-decrypt-later target that the post-quantum channel of §6 protects for auditor disclosure but would not protect here, and it is personal data written to an immutable medium, which is a poor posture under the Brazilian data-protection law.
3. The paying institution makes the instant payment; the PSP confirms; the E2EID is resolved.
4. The PSP signs an EIP-712 receipt over $(c_0, \nu, \text{pc}, \text{settled}, \text{deadline})$, recomputing c_0 from the amount it settled and `pc` from the debtor identifier in the payment message; the oracle attests it (§4).
5. The proof π is generated over the public signals $(c_0, c, \nu, K, \text{pc})$ and `finalize( $\pi$ )` verifies it, cross-checks the oracle attestation for ν (same c_0 and same `pc`), checks the domain anchor K , consumes the tag, and transitions the operation to FINALIZED. It becomes IRREVOCABLE only after the rail’s refund window elapses without a reversal (§4).

4 The Attestation Oracle

The circuit proves *knowledge* of a preimage; it does *not*, on its own, prove that the E2EID corresponds to a payment actually settled at the SPI. Historically this was resolved purely off-chain: the PSP confirms the payment, returns the E2EID, and the paying institution submits it — an unverified claim. As §2 notes, an E2EID exists for rejected payments too, so the claim is not even self-evidencing.

4.1 A receipt that does not carry the value

The oracle makes the trust link explicit and on-chain verifiable. A trusted PSP (the *attestor*) signs, off-chain, an EIP-712 receipt

$$\text{Receipt}(\underbrace{c_0}_{\text{bytes32}}, \underbrace{\nu}_{\text{settlement tag}}, \underbrace{\text{pc}}_{\text{counterparty commitment}}, \underbrace{\text{settled}}_{\text{bool}}, \underbrace{\text{deadline}}_{\text{uint64}}),$$

and any relayer submits it via `attest(receipt, signature)`.

The choice of fields matters because the naive alternative genuinely leaks. Consider signing `Receipt(txid,  $\nu$ , amount, settled)` instead: since the contract must recompute the EIP-712 digest to recover the signer, every signed field necessarily appears *in the clear in the calldata* of `attest`, which is public and permanent — and a naive implementation is likely to also emit the amount in an event, for indexing. Since ν is the same tag the settlement circuit exposes, any observer could join the public settlement (c, ν) to that attestation and read off the amount. Such a design would falsify the confidentiality claim of §8 and Property 5, and the amount would not even be used in any on-chain check — pure liability for no benefit.

Signing over c_0 avoids the leak and, unlike signing the amount directly, *adds* a check rather than a mere pass-through. The PSP holds salt_0 and txid from charge creation and observes the amount it actually settled; it recomputes c_0 from those three values. **This recomputation is normative, not an implementation detail: the PSP must evaluate the Poseidon itself — for c_0 over the amount it observed settling, for pc over the debtor identifier in the payment message, and for the tag $\nu = \text{Poseidon}(\text{e2eld}, k)$ over the E2EID it observed, using the domain key it holds — and must never sign a c_0 , pc or ν value received from the institution.** A PSP whose API merely echoes a client-supplied commitment turns the value-consistency check into a tautology — the signature would attest that the client’s number equals the client’s number — and a client-supplied tag is worse: it would let the institution attach an arbitrary tag to a real payment, severing the anti-replay binding between ν and the E2EID, unless the PSP kept its own $\text{e2eld} \rightarrow$ attested registry, which is exactly the state this design avoids. This is the same failure shape as the vacuous verification verdict of §9.3, one layer up, and an integration audit should test for it explicitly.

A payment for the wrong amount produces a different c_0 , the receipt then refers to a commitment the escrow is not locked on, and the settlement does not close. The protocol thus verifies *value consistency without the value appearing anywhere on-chain*. The **deadline** bounds the window in which a signed receipt can be used, containing the damage from a compromised PSP key. The receipt additionally carries pc , the counterparty commitment of §3, which the PSP computes over the debtor identifier in the payment message using the salt_p shared at charge creation. This adds nothing to what an observer sees: pc is already a public signal of the settlement proof, it is a Poseidon image under a ≈ 254 -bit salt, and the attestation and the settlement were already linkable through ν . What it adds is a check: **finalize** accepts only if the attested pc equals the proof’s public signal, so the counterparty the compliance predicates speak about is the one the PSP saw paying — not one the prover chose.

The contract recovers the signer with **ECDSA.recover** over the EIP-712 digest and accepts *iff*: the signer is a registered attester; **settled** = true; the receipt has not expired; and ν has never been attested (anti-replay). On success it records ($\nu \mapsto \text{signer}, c_0, \text{pc}, \text{timestamp}$) — in the reference implementation c_0 and pc are stored as the single word $\text{keccak256}(c_0 \parallel \text{pc})$, which keeps the storage footprint of the attestation unchanged — and emits an event that carries *no* amount, no charge identifier and no party identifier. The PSP is gasless: it only signs; the relayer pays gas and holds no power to forge, since acceptance is gated on the PSP signature.

Two consequences of sharing the blinding factors deserve stating. Against the PSP itself, c_0 and pc are not hiding at all — by design: the check is the PSP recomputing them, and since the PSP separately holds every other preimage it processed (amount, txid , e2eld , party, and the salts), it is, by construction, a complete linkability oracle for the operations that pass through it: nothing in this protocol hides the fiat leg from the PSP that services it (§8 states this as a scope boundary of the confidentiality claim, not an oversight). Against any third party that obtains salt_0 (or salt_p), the hiding of c_0 (or pc) degrades from the salt’s ≈ 254 bits to the entropy of the remaining preimage: for pc that is the CPF/CNPJ space, enumerable outright; for c_0 it is (amount, txid), enumerable only if the charge id — 26 to 35 alphanumeric characters — leaks through the same channel, which in practice it does. The salts are therefore secrets of the same sensitivity as the payment data they blind, and the off-chain channel of §3 step 2 must protect them accordingly. And since **attest** is write-once per tag, a receipt the PSP signs over a *wrong* recomputed c_0 — a software bug, not an attack — permanently burns that ν : the payment can no longer close *that* operation, and the c_0 it was locked on falls back on the liveness timeout of §4 (**cancel**, once the **lock** deadline passes) rather than staying stuck. We state this as a property of the anti-replay choice rather than discover it in production.

4.2 What settled means, and reversal

Settlement at the SPI is not the end of the story. The rail provides a return mechanism (certain Pix return/fraud-recovery mechanisms may remain effective for up to 90 days), a dedicated fraud-refund mechanism, and the receiving PSP may place a precautionary hold of up to 72 hours. We therefore define, as an internal semantic of this protocol’s `settled` field — not a redefinition of the rail’s own settlement terminology — that `settled = true` means *credit available to the receiver*, not merely “cleared at the SPI”; this is what the PSP must observe before it may sign a receipt with `settled = true` (§4).

Accordingly the oracle exposes `attestReversal`, in which the *same* attester signs (ν , `returned = true`, `deadline`) within a configured dispute window, and the operation state machine distinguishes `FINALIZED` from `IRREVOCABLE`: the latter is reached only once the window elapses with no reversal. **Read Irrevocable as a protocol-state name, not an economic-finality claim:** it means only that *this contract* will no longer accept a reversal for this operation — which, as the very next paragraph quantifies, is not the same as the rail being unable to reverse the underlying payment for the following ≈ 87 days. A delivery-versus-payment flow must gate the irreversible release of the asset on `isIrrevocable`, not on `isAttested`, while still pricing in that residual rail-level risk separately. Revoking an attester blocks *new* attestations and does not retroactively invalidate past ones; retroactive invalidation would hand the registry owner the power to undo completed settlements.

The state machine is deliberately open to one orthogonal upgrade: a `RailAnchored` flag, settable at any time by a verified rail-level statement (§8, path iv), which strengthens the evidence behind a `FINALIZED` or `IRREVOCABLE` operation — and, applied to returns, narrows the post-window residual above from invisible to visible ex post — without ever sitting on the settlement’s critical path.

One trade-off must be stated numerically rather than implied. The dispute window is a deployment parameter, and the reference deployment sets it to 72 hours. That figure mirrors the rail’s precautionary-hold period, but under the credit-available semantics above it corresponds to *no* rail deadline: a hold is resolved before the PSP attests, and the mechanisms that can still reverse a credited payment — the ordinary return, which may remain effective for up to 90 days, and the dedicated fraud-refund mechanism, on its own separate track — extend far beyond it. The 72-hour window therefore catches early returns only. An operation that reaches `IRREVOCABLE` after 72 hours remains exposed, for the remaining ≈ 87 days, to a rail-level refund that the on-chain state machine will not represent; that residual risk sits with the asset leg, and a deployment accepts it as the price of not locking capital for three months. A deployment that instead requires `IRREVOCABLE` to dominate the refund window must set the window to 90 days and carry the capital cost. Both are defensible; choosing silently is not, so the reference artifacts choose 72 hours and say so.

5 Confidential Compliance Predicates

Beyond proving that fiat settled, a regulated deployment must answer questions about a transaction *without opening it*. We extend the commitment scheme with three ZK *policy predicates*, each a small Groth16 circuit whose only public signals are commitments and a policy anchor; the sensitive data never appears, *not even to the auditor*. We call them policy predicates rather than compliance predicates deliberately: each proves membership or a bound against an on-chain policy anchor set by a named authority (§5, “Policy authority on-chain”), which is necessary input to a compliance determination but does not itself constitute one — amount-within-limit is a policy condition, not a legal category, and the two membership predicates below are named to make the same point explicit.

- **Value within limit** (`compliance_range_verify`): proves amount $<$ limit for an opening of the commitment, via a bounded comparison (`Num2Bits(64)` range-checking both operands, then `LessThan`).
- **Non-membership in the attested sanctions set** (`sanctions_exclusion_verify`): proves the counterparty, committed as `pc`, is *absent* from a Sparse Merkle Tree of sanctioned parties (circomlib `SMTVerifier` non-membership, `fnc = 1`). We name it this way, not “sanctions clearance”, deliberately: the predicate proves absence from *the specific set the anchored root represents*, at the root’s version, under whatever normalization built it (§5, Open operational questions, item i) — it is not equivalent to a legal determination that the counterparty is unsanctioned, which depends on list freshness, normalization and coverage the circuit has no visibility into.
- **KYC-set membership** (`kyc_inclusion_verify`): the mirror predicate — proves the counterparty is *present* in an allow-list SMT of identities the root’s operator has verified (`fnc = 0`). Likewise not “KYC compliance”: that determination is delegated entirely to whichever authority admits identities into the root and governs its lifecycle (who verified, when, under what jurisdiction, whether revoked) — the predicate proves set membership, nothing about the verification behind it.

Whose counterparty? A predicate about a party the prover chose is worthless. If the counterparty commitment were an independent input to each predicate circuit, a malicious institution could commit to any clean identity and prove exclusion and inclusion about *that* identity while settling a payment involving someone else — with no counterparty field on the settlement side to contradict it. The construction closes this structurally, with two halves: `pc` is a public signal of the settlement circuit (§3), so settlement and predicates provably speak about the same committed party; and `pc` is a signed field of the PSP receipt (§4), computed over the debtor identifier in the payment message, with `finalize` requiring equality between the attested value and the proof’s public signal. The predicates consume that same `pc`. The counterparty they speak about is then, by construction, the counterparty that paid — under Assumption (I1): a PSP that signs a wrong `pc` is the same trusted-PSP residue as a PSP that signs a false `settled`.

Policy authority on-chain. The policy anchor (the `limit`, or the `sanctionsRoot/kycRoot`) must be fixed by an on-chain policy authority, not supplied by the prover. Otherwise a malicious institution could prove compliance against a favorable self-chosen limit, or exclusion against an empty sanctions tree. The attestors pin the official policy (`setPolicyLimit/setKycRoot`, guarded by a `policyAdmin`) and reject any proof whose public policy signal does not equal it. We validated on-chain that a *genuinely valid* Groth16 proof for a self-chosen loose limit is rejected precisely because it does not match the official policy.

Open operational questions. We flag three that the cryptography does not settle and a deployment must. (i) *List normalization*: sanctions lists are keyed differently — OFAC matches on names and aliases, the Brazilian CEIS/CNEP registries on CPF/CNPJ. An SMT over identifiers cannot express name-matching semantics, so a deployment must fix a normalization and accept the resulting false-negative profile explicitly. (ii) *Root lifecycle*: who updates `sanctionsRoot`, how often, and — the sharp question — whether proofs issued against a superseded root remain valid. Both answers are defensible; leaving it implicit is not. (iii) *Per-transaction limit is not AML*: the range predicate bounds a single transaction and says nothing about structuring across many. Aggregate

monitoring is a different problem and we do not claim to address it. (iv) *Whose identity, precisely:* naming this precisely matters more than it looks, so we fix three terms rather than use “counterparty” loosely. `payerId` is the debtor account holder exactly as named in the payment message — what `pc` actually commits. The *economicCounterpartyId* is whoever the transaction is economically for, which coincides with `payerId` except when a third party pays on the debtor’s behalf. The *beneficialOwnerId* is a further AML/KYC concept (who ultimately controls or benefits, e.g. behind a shell entity) that this construction does not represent at all. Precisely stated, then: the policy predicates of this section prove a predicate about `payerId`, always; a deployment must decide whether third-party payments are rejected at attestation (so `payerId = economicCounterpartyId` always holds), or accepted with `pc` understood as “who actually paid” — the more defensible question for AML purposes, but a different question from “who is the beneficial owner”, which this construction leaves entirely to the PSP’s own KYC process and does not attempt to answer.

6 Post-Quantum Audit Disclosure: A Preliminary Construction

This section is an optional, crypto-agile audit-disclosure channel and is not part of the core Proof-of-Instant-Settlement security argument (§7–§8): none of the properties or the threat-model rows there depend on anything in this section, and a deployment can run the settlement construction with only the classical (X25519) channel selected. We report it here as a preliminary, sized construction, not as a claim the core contribution rests on.

Auditor disclosure has *long-lived* confidentiality: a ciphertext recorded today must resist a future quantum adversary (“harvest now, decrypt later”). The disclosure channel is therefore crypto-agile and selectable per deployment — classical (X25519), post-quantum (ML-KEM-768, FIPS-203 [9]), or a hybrid of the two — declared on-chain (`auditChannelKind`, default hybrid) and, in the intended deployment, changeable only through consortium governance (a timelocked proposal, not an administrator setter). The reference artifacts, being a single-maintainer prototype, stand a single admin key in for that governance — the same honesty caveat as the registry owner in §8 — so the governance property should be read as *designed*, not as *demonstrated*. Each auditor publishes its own encryption key, so no administrator can redirect it to intercept disclosures.

In-circuit binding. To make a disclosure *verifiable by anyone without decrypting*, we prove in zero knowledge that the published ciphertext encrypts the opening of the commitment under a key derived from the KEM shared secret. Following Qurrency [10] — which uses the *same* Groth16/circom/Poseidon /BN254 stack — we port its open-source (Apache-2.0) ML-KEM-in-circom implementation and connect `mlkem_encaps` to our commitment. The resulting circuit `audit_binding_pq_verify` compiles at $\approx 1.62\text{M}$ constraints; the shared secret is thereby *proven* to originate from ML-KEM, not merely asserted. A symmetric-only binding (shared secret as a private input) is validated on-chain in the interim; the trusted-setup ceremony for the full 1.62M-constraint circuit requires production hardware ($\sim 16\text{ GB RAM}$, 2^{21} powers of tau) and is deferred to the pilot.

Status (preliminary). We report this post-quantum binding as *preliminary*. Unlike the settlement and predicate circuits — for which we give proving time and on-chain verification gas — the full in-circuit ML-KEM binding has no end-to-end proving-time or on-chain figure yet, pending the production-scale setup. What is established is that it *compiles and is sized* on our exact stack; the selectable channel and the symmetric binding are validated; the full binding is work in progress rather than a deployed result.

7 Formal Model and Security Properties

We give a syntax for the construction and state its security properties under explicit cryptographic, operational, institutional and governance assumptions — the table below (A1)–(A4) names which assumption is which kind; not every property that follows reduces to a standard cryptographic assumption alone, and we say so at each one rather than let “formal properties” imply otherwise. Let $\Pi = (\text{Setup}, \text{Commit}, \text{Prove}, \text{Verify}, \text{Attest}, \text{Settle})$ over the BN254 scalar field $\mathbb{F}_p$:

- $\text{SetupZK}(1^\lambda) \rightarrow (\text{crs}, \text{vk})$: the per-circuit Groth16 common reference string and verification key.
- $\text{SetupDomain}(1^\lambda) \rightarrow (k, K)$: a fresh domain key k and its on-chain anchor $K = \text{Poseidon}(k)$, registered at domain creation and immutable thereafter (§3). The two setups are independent: a domain can be created without a new ceremony, and the ceremony says nothing about any domain’s key.
- $\text{Commit}_0(\text{amount}, \text{salt}_0, \text{txid}) \rightarrow c_0$ and $\text{Commit}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}) \rightarrow c$.
- $\text{Prove}(\text{crs}, w) \rightarrow (\pi, \nu)$ with witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ and tag $\nu = \text{Poseidon}(\text{e2eld}, k)$.
- $\text{Verify}(\text{vk}, c_0, c, \nu, K, \text{pc}, \pi) \rightarrow \{0, 1\}$.
- **Attest**: a registered PSP signs (EIP-712) a receipt $\text{Receipt}(c_0, \nu, \text{pc}, \text{true}, \text{deadline})$, recomputing c_0 and pc itself (§4).
- **Settle**: accept iff $\text{Verify} = 1$, the receipt signature verifies against a registered attestor, the receipt is unexpired, its c_0 is the one the escrow locked on, its pc equals the proof’s public counterparty commitment, and the tag is unused.

We assume: **(A1)** Groth16 is knowledge-sound and zero-knowledge under its trusted setup [1]; **(A2)** Poseidon is collision-resistant and, on an input with a high-entropy component, hiding (modeled as a random oracle) [2]; **(A3)** ECDSA/EIP-712 is EUF-CMA; **(A4)** within a domain, txid uniquely identifies the agreed payment obligation and is not legitimately reused for two distinct charges (Charge-identifier uniqueness); **(A5)** keccak256 is collision-resistant for the binding hash the attestation oracle computes over (c_0, pc) (§4). (A4) is what lets txid do the anti-cross-charge work Equations (3)–(4) rely on (§3); it is imposed operationally by the charge-creation flow (the orchestrator mints a fresh txid per charge and the PSP’s own charge API rejects a duplicate), not proved by the circuit, which only checks that *the same* witness txid appears in both commitments. (A5) is stated separately from (A2) because the two hash functions are used for different purposes and neither collision-resistance assumption implies the other: Poseidon binds circuit-internal witness values, while keccak256 binds the oracle’s off-circuit (c_0, pc) pair to a single ν (Theorem 2). The assumptions above are not all the same *kind* of assumption, and treating them as if they were is itself a source of overclaiming:

Assumption	Nature	Failure mode if false
(A1) Groth16 sound./ZK	cryptographic	forged proofs / leaked witness
(A2) Poseidon coll.-res.	cryptographic	two openings of one commitment
(A2) Poseidon hiding	heuristic (RO model)	preimage recovery, if the model is wrong
(A3) ECDSA/EIP-712 EUF-CMA	cryptographic	forged attestations
(A4) <code>txid</code> uniqueness	operational	cross-charge proof reuse
(A5) Keccak-256 coll.-res.	cryptographic	oracle binding forged (pc substitution)
(I1) Trusted PSP	institutional	false <code>settled</code> accepted
(S1) Trusted setup	cryptographic, contingent on process	forged proofs, if toxic waste leaked
Policy-root correctness	governance	wrong predicate answers accepted

The cryptographic rows are the ones a reduction can be stated against; the operational, institutional and governance rows cannot be discharged by better cryptography, only by better process, and no amount of tightening the ZK layer changes that.

Property 1 (Correctness). *For an honestly generated $(c_0, c, \nu, K, \text{pc}, \pi)$ of a settled payment with a valid, unexpired PSP receipt, `Settle` accepts.*

The next property is stated in three parts on purpose: binding, two-phase consistency and settlement binding are three different guarantees that a single “operation binding” statement would blur together, and a reviewer should be able to attack each independently.

Property 2 (2A — Commitment binding). *No PPT adversary produces two distinct openings of the same c (or of c_0), except with negligible probability.*

Proof sketch. Two distinct openings of one Poseidon image are a collision, contradicting (A2) directly; this holds independent of Groth16 or of any protocol logic above it. $\square$

Property 3 (2B — Two-phase consistency). *For any accepting proof π for public inputs $(c_0, c, \nu, K, \text{pc})$, the amount and `txid` used to construct c are identical to those used to construct c_0 .*

Theorem 1 (Two-phase consistency). *Assuming the knowledge soundness of Groth16 (A1), except with negligible probability, every accepting proof admits a witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ satisfying the circuit relation of §3 (Equations 3–7). In that relation, the same witness values `amount` and `txid` occur in both commitment equations — Equation (3), $c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid})$, and Equation (4), $c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid})$. Consequently, an accepting proof cannot represent a transition in which the amount or `txid` used for the post-payment commitment c differs from the amount or `txid` committed at escrow creation, i.e. in c_0 .*

Proof. Let π be an accepting Groth16 proof. By knowledge soundness (A1), except with negligible probability, there exists an extracted witness w satisfying the circuit relation. The circuit contains a single witness variable `amount`, used in both Equation (3) and Equation (4), and a single witness variable `txid`, likewise used in both. There are therefore no independent values $\text{amount}_0, \text{amount}_1$ or $\text{txid}_0, \text{txid}_1$ that could be chosen separately for c_0 and c : the circuit relation does not admit them. Thus, for every accepting witness, $\text{amount}_{c_0} = \text{amount}_c = \text{amount}$ and $\text{txid}_{c_0} = \text{txid}_c = \text{txid}$. A proof for which either value differed between the two phases would require a witness satisfying a circuit relation the circuit does not define; by knowledge soundness, no such witness can be extracted from an accepting proof except with negligible probability. Therefore,

$$\Pr[\text{Bad}_{2B}(\mathcal{A})] \leq \text{Adv}_{\text{Groth16}}^{\text{KS}}(\mathcal{A}), \quad (8)$$

where $\text{Bad}_{2B}(\mathcal{A})$ is the event that $\mathcal{A}$ produces an accepting proof whose extracted witness assigns different amount or `txid` values to c_0 and c . $\square$

Corollary 1 (Escrow-to-settlement term consistency). *If an escrow is created against c_0 , any subsequently accepted proof for that escrow necessarily uses the same amount and txid in its post-payment commitment c . The property does not by itself establish that either commitment corresponds to a real-world payment; that semantic fact remains outside the cryptographic relation and is handled by the PSP attestation (Property 7) and the Trusted-PSP assumption (Assumption (I1)).*

Property 4 (2C — Authenticated settlement binding). *Let D be a domain with registered anchor $K_D = \text{Poseidon}(k_D)$, where k_D is the domain secret; elsewhere in this paper, where a single domain is fixed throughout, we write K , k without the subscript, and K_D , k_D specialize to exactly those when D is held fixed. Let an adversary $\mathcal{A}$ interact with the public settlement system and submit arbitrary proofs and receipts to the attestation oracle and **Settle**.*

An accepted settlement is authentically settlement-bound if, except with negligible probability, there exist a registered attestor P ; an EIP-712 receipt $R = (c_0, \nu, \text{pc}, \text{true}, \text{deadline})$; a valid PSP signature $\sigma = \text{Sign}_P(R)$; and a Groth16 witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ satisfying the circuit constraints of §3 (Equations 3–7), restated here for convenience:

$$\begin{aligned} c_0 &= \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}), & c &= \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}), \\ \nu &= \text{Poseidon}(\text{e2eld}, k), & K_D &= \text{Poseidon}(k), \\ \text{pc} &= \text{Poseidon}(\text{party}, \text{salt}_p), \end{aligned}$$

*and, moreover, the receipt authenticated by P is bound to exactly the same c_0 , ν , and pc accepted by **Settle**.*

*This property is an authentication and cryptographic-consistency property. It does **not** assert that the PSP’s statement `settled = true` is truthful. The latter remains Assumption (I1).*

Theorem 2 (Settlement-binding theorem). *Conditioned on the deterministic correctness of the deployed **Settle** contract with respect to the acceptance conditions specified in §3 — a property of the deployed code, not a cryptographic hypothesis, and hence not itself a term in the bound below — and on the registered domain anchor K_D being immutable during the lifetime of domain D (§3), assume further: (1) Groth16 is knowledge-sound for the deployed `settlement_verify_v2` circuit; (2) Poseidon is collision-resistant over the BN254 field; (3) ECDSA/EIP-712 is EUF-CMA secure; (4) keccak256 is collision-resistant for the oracle’s binding hash — exactly (A1)–(A3) and (A5) above, gathered here so the theorem’s cryptographic hypotheses are self-contained rather than scattered.*

*Then, for every probabilistic polynomial-time adversary $\mathcal{A}$, the probability that $\mathcal{A}$ causes **Settle** to accept a settlement for which either (a) no registered PSP signed the accepted tuple $(c_0, \nu, \text{pc}, \text{true}, \text{deadline})$, or (b) the accepted ν is not equal to $\text{Poseidon}(\text{e2eld}, k_D)$ for the `e2eld` contained in an accepting witness for the registered domain, is negligible. More precisely,*

$$\Pr[\text{Bad}_{SB}(\mathcal{A})] \leq \text{Adv}_{\text{ECDSA}}^{\text{EUF-CMA}}(\mathcal{A}) + \text{Adv}_{\text{Groth16}}^{\text{KS}}(\mathcal{A}) + \text{Adv}_{\text{Poseidon}}^{\text{CR}}(\mathcal{A}) + \text{Adv}_{\text{Keccak}}^{\text{CR}}(\mathcal{A}), \quad (9)$$

where each term on the right is the standard cryptographic advantage against the named primitive, and the bound holds under the deterministic conditioning stated above — a contract-correctness failure is a software bug outside the reduction, not a probabilistic event the reduction accounts for.

The theorem binds ν to the `e2eld` contained in the accepting ZK witness; it does **not**, by itself, establish that the PSP observed that same `e2eld`. That latter correspondence is part of the Trusted-PSP assumption (I1).

Proof. Suppose $\mathcal{A}$ causes **Settle** to accept an operation. By the acceptance condition of **Settle**, the submitted Groth16 proof verifies against the public tuple $(c_0, c, \nu, K_D, \text{pc})$. By Groth16 knowledge

soundness, except with negligible probability there exists an extractor obtaining a witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ satisfying every circuit constraint. In particular, the witness satisfies $\nu = \text{Poseidon}(\text{e2eld}, k)$ and $K_D = \text{Poseidon}(k)$ — precisely Equations (5) and (6) of §3.

Because K_D is the immutable anchor registered for domain D , and because the domain key was generated as $K_D = \text{Poseidon}(k_D)$, we have both $\text{Poseidon}(k) = K_D = \text{Poseidon}(k_D)$. If $k \neq k_D$, these are two distinct preimages of the same Poseidon output; under Poseidon collision resistance this occurs only with negligible probability. Therefore, except with negligible probability, $k = k_D$. Substituting into (5) gives $\nu = \text{Poseidon}(\text{e2eld}, k_D)$: an accepted proof cannot use an arbitrary domain key or an independently chosen settlement tag without either violating Groth16 soundness or finding a Poseidon collision.

It remains to establish that the accepted c_0 , ν , and pc are the values authenticated by the PSP. The attestation oracle accepts a receipt only after recovering the signer from the EIP-712 digest and checking that the signer is a registered attestor, that $\text{settled} = \text{true}$, that the receipt is unexpired, and that ν has not previously been attested. The oracle stores the binding associated with ν as $\text{keccak256}(c_0 \parallel \text{pc})$. Settle subsequently requires that the oracle’s stored binding for the submitted ν equal $\text{keccak256}(c_0 \parallel \text{pc})$ for the c_0 locked by the escrow and the pc supplied as the proof’s public signal.

Therefore, if $\mathcal{A}$ presents an accepted settlement for which no registered PSP signed the exact tuple $(c_0, \nu, \text{pc}, \text{true}, \text{deadline})$, then either (i) the EIP-712 signature was forged or altered, contradicting EUF-CMA security (A3); or (ii) the contract/oracle accepted a tuple not authenticated by the registered attestor, which is excluded by the theorem’s conditioning on deterministic contract correctness rather than by a cryptographic reduction — (ii) is a contract-correctness failure, not a probabilistic event this bound accounts for. Similarly, if the accepted pc differs from the PSP-attested pc' , the oracle binding check would require $\text{keccak256}(c_0 \parallel \text{pc}) = \text{keccak256}(c_0 \parallel \text{pc}')$ for $\text{pc} \neq \text{pc}'$, except with negligible probability under (A5).

Consequently, except with negligible probability, every accepted settlement contains $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ for the e2eld in the accepting witness, and the same ν , c_0 , and pc are authenticated by a registered PSP receipt. This proves authenticated settlement binding. $\square$

Corollary 2 (Counterparty binding). *Under the hypotheses of Theorem 2, an accepted settlement cannot substitute an arbitrary counterparty commitment pc' for the commitment pc authenticated by the PSP, except with negligible probability. Therefore, any compliance predicate of §5 subsequently evaluated over the public pc is bound to the same counterparty commitment the PSP authenticated for the settlement. This does not establish that the PSP correctly identified the payer — that semantic claim remains part of Assumption (I1).*

Security-boundary note. Theorem 2 deliberately does not claim that the PSP’s statement $\text{settled} = \text{true}$ is objectively true at the payment rail. It establishes only that (1) a registered PSP authenticated the statement; (2) the authenticated statement refers to the same c_0 , ν , and pc used by the accepted settlement; and (3) the accepted ν is cryptographically bound to the e2eld in the Groth16 witness under the registered domain key. The implication “PSP signed $\text{settled} = \text{true} \Rightarrow$ payment actually settled” is therefore intentionally outside the theorem and remains the institutional Trusted-PSP assumption of the construction (Assumption (I1)).

Properties 2 and 3 are unconditional under (A1)–(A2). Property 4 is not: it routes through Assumption (I1) for what the attestation *means*, even though *that a registered key produced it* is unconditional under (A3) — exactly the authentication/truthfulness split Property 7 states.

Property 5 (Confidentiality of settlement data under the Poseidon random-oracle model). *For any PPT adversary observing the public transcript — $(c_0, c, \nu, K, \text{pc}, \pi)$ together with the calldata of `attest` and `settle`, i.e. including `Receipt` $(c_0, \nu, \text{pc}, \text{settled}, \text{deadline})$ and its valid EIP-712 signature under a registered attester key — the adversary’s advantage in recovering $(\text{amount}, \text{e2eld}, \text{party}, \text{txid})$ is negligible. The property is scoped to on-chain observers without access to the off-chain secrets $(\text{salt}_0, \text{salt}, \text{salt}_p, k)$: the attesting PSP is explicitly outside this scope, since it holds those secrets itself (§4 states this as a linkability boundary, not an oversight). This is a hiding claim about the four excluded values, not a claim that the whole transcript — signature included — can be produced by a simulator lacking the attester’s signing key: no such simulator is claimed or needed, since the adversary here is given a genuine transcript, not asked to distinguish one from a simulation of it. Signature authenticity is a separate matter, stated and proved as Property 7 under (A3); this property does not depend on it and does not restate it. The hiding argument below is a random-oracle heuristic for Poseidon (A2), not a reduction to a standalone computational hardness assumption; we state it as a heuristic explicitly, which is the standard treatment for a permutation-based hash used as a commitment, and do not claim a tighter reduction than that.*

Proof sketch, in four parts. The transcript is stated to include the receipt, its signature, and the calldata deliberately: a transcript restricted to (c, ν, π) alone would be insufficient — any receipt design whose signed fields carry the amount or `txid` in the clear, as §4 discusses, falsifies the property outright, since that calldata is exactly as public and permanent as the settlement transcript itself. We argue each public element’s contribution to the adversary’s advantage independently, which suffices because the elements below are exactly the transcript’s public elements and none is a function of the others computable without the hidden witness. (A) *ZK privacy of π* . Groth16 zero-knowledge (A1) gives a simulator for π alone, given only the public signals $(c_0, c, \nu, K, \text{pc})$; this is the standard Groth16 guarantee and needs no extension here. (B) *Commitment hiding of c_0, c, pc, ν* . Each is a Poseidon image with a high-entropy blinding component under (A2)’s random-oracle heuristic: c_0 and c under ≈ 254 -bit salts (field elements of the BN254 scalar field), pc under salt_p , and ν under the ≈ 254 -bit domain key k . $K = \text{Poseidon}(k)$ is a commitment to that same high-entropy key and is constant per domain, so it carries no *per-operation* information beyond identifying the domain. We do not claim these four images are mutually independent as random variables — c_0 and c share the witness values *amount* and *txid* (Property 3), and ν and K share k — only that each is individually hiding. Under the random-oracle heuristic, each commitment image is computationally hiding with respect to its high-entropy blinding component; under this heuristic, the stated transcript does not provide a feasible recovery strategy for the excluded preimages beyond the explicitly excluded side information. (C) *Metadata is explicitly out of scope, not implicitly covered*. This property says nothing about transaction existence, timing, on-chain addresses (submitter, relayer, attester) or cross-operation linkability from a repeated `pc` or K — §8’s “confidentiality is not unlinkability” table states exactly which of these are and are not protected, and none of the protected ones is this property’s subject. (D) *The signature adds authentication, and nothing an adversary can extract*. We do not claim the signature bits themselves are simulatable — a PPT adversary lacking the attester’s key cannot produce one, which is precisely EUF-CMA (A3), not a hiding property, and is exactly why it is Property 7’s claim, not this one’s. What this property needs, and gets, is narrower: the message `Receipt` $(c_0, \nu, \text{pc}, \text{settled}, \text{deadline})$ the adversary sees signed contains no plaintext of $(\text{amount}, \text{e2eld}, \text{party}, \text{txid})$ — every field in it is either already covered by (B) or a public boolean/timestamp — so *observing a valid signature over that message* gives the adversary nothing toward recovering the four excluded values beyond what (B) already accounts for. Composing (A)–(D): every source of information in the stated transcript either carries negligible advantage toward the four excluded values (A, B) or is explicitly out of

this property’s scope (C), and the signature’s own unforgeability (D) is a fact about who signed, orthogonal to what the signed message reveals. $\square$

Property 6 (Domain-scoped settlement uniqueness). *Within a domain under a fixed, immutable key K , and under Assumption (I1)’s $\text{PSP } \text{e2eld} \rightarrow \text{txid}$ bijection, each settled payment closes at most one operation. The property is explicitly domain-scoped: it neither states nor implies uniqueness across domains (distinct K , including a rotated one) without that same assumption.*

Theorem 3 (Domain-scoped settlement uniqueness). *For a fixed domain anchor K_D , assuming domain-key immutability (§3), collision resistance of Poseidon (A2), single-use enforcement of ν by the oracle and by Settle (§4, §3), and the PSP’s $\text{e2eld} \rightarrow \text{txid}$ uniqueness assumption (Assumption (I1)), no PPT adversary can cause two accepted settlements corresponding to the same e2eld , except with negligible probability.*

Proof. The proof follows the same four steps as the tag argument of Theorem 2, specialized to two accepted settlements rather than one.

- (i) The same e2eld under the same domain key k_D produces the same tag: $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ is a deterministic function of e2eld once k_D is fixed (Equation (5) of §3).
- (ii) K_D forces the same key across both proofs: by the collision argument of Theorem 2, any proof accepted under the public anchor K_D extracts a witness key k with $\text{Poseidon}(k) = K_D$, and except with negligible Poseidon-collision probability, $k = k_D$ for *every* such proof — so two settlements in the same domain cannot use two different domain keys.
- (iii) The registry marks ν as used on first acceptance: on the first accepted settlement for this e2eld , Settle records ν as spent (§3).
- (iv) A second settlement for the same e2eld presents the same ν (by (i)–(ii)) and is rejected, since Settle requires an unused tag (iii).

Therefore, except with negligible probability, no two accepted settlements in the same domain correspond to the same e2eld . Cross-domain uniqueness (distinct K_D , including a rotated one) is not implied by this argument — a fresh domain admits a fresh k_D and hence a fresh ν for the same e2eld — and rests entirely on the PSP’s $\text{e2eld} \rightarrow \text{txid}$ bijection, Assumption (I1). $\square$

Property 7 (Attestation unforgeability). *Settle accepts only if a registered PSP signed an unexpired receipt with $\text{settled} = \text{true}$ for the operation’s c_0 and for the proof’s counterparty commitment pc . This property is authentication of the attestation, i.e. that a registered key produced it — it is not truthfulness of the underlying settlement claim, which is exactly the residual trust of Assumption (I1).*

Proof sketch. Acceptance recovers the signer via `ECDsa.Recover` over the EIP-712 digest (domain-separated by `chainId` and the oracle address) and checks registration and expiry; a forgery breaks EUF-CMA (A3). This does *not* constrain a *malicious registered* PSP — the residual Trusted-PSP assumption. $\square$

Soundness (Properties 2–4 and 7) holds only under (A1)’s trusted-setup premise, which the reference artifacts meet at development grade only (the trusted-setup assumption of §8). We now enumerate adversaries informally, including those outside the formal statements above.

8 Security Analysis and Threat Model

We enumerate adversaries and state, for each, whether the construction defends against it.

Adversary / attack	Mitigation (or explicit gap)
Paying institution forges an E2EID it never paid	Mitigated by the oracle: acceptance requires a PSP signature over <code>settled=true</code> . Note that an E2EID exists for rejected payments too (§2), so the identifier alone proves nothing.
Replay: reuse one settled instant payment to close two operations	Mitigated : the tag ν is single-use, enforced both at the oracle (<code>attest</code>) and at settlement; the anchor K forces one k per domain.
Deanonymization: recover <code>e2eld</code> from the public tag	Mitigated by keying the tag with a ≈ 254 -bit domain secret. The threat is not only brute force: a party holding the E2EID legitimately (the payer’s bank, not a party to the operation) could otherwise match by hashing.
Proof reuse across charges	Mitigated : <code>txid</code> is bound inside both c_0 and c , under (A4) that <code>txid</code> is not legitimately reused across charges.
Settled amount differs from the agreed amount	Mitigated : the PSP recomputes c_0 from the amount it settled; a mismatch yields a c_0 the escrow is not locked on.
Confidentiality: learning amount/parties from chain data	Mitigated : every public element (c_0 , c , ν , K , <code>pc</code> , and the receipt) is a Poseidon image or a proof. This was <i>not</i> the case while the receipt carried the amount (§4).
Prover picks a clean counterparty for the compliance predicates	Mitigated : <code>pc</code> is a public signal of the settlement circuit and a signed field of the PSP receipt, computed by the PSP over the payer identifier in the payment message; <code>finalize</code> requires the attested <code>pc</code> to equal the proof’s public signal. A PSP that signs a wrong <code>pc</code> is Assumption (I1).
A receipt is attested with a wrong c_0 or <code>pc</code> — a PSP software bug, <i>or</i> a compromised/malicious PSP deliberately grieving, since the same action a bug produces also destroys liveness on demand	Tag burned, operation recoverable via timeout : <code>attest</code> is write-once per ν , so that specific ν cannot be re-attested, but <code>cancel</code> (§4) lets the operation’s c_0 expire once its <code>lock</code> deadline passes, independent of the PSP. This is a deliberate property of anti-replay, not a bug: a PSP holding the attestor key has a standing <i>griefing</i> capability (destroy liveness on any operation it is asked to attest for) that a receipt-signature forgery does not grant it a stronger one from; <code>cancel</code> bounds the damage to one missed deadline, not a permanently stuck escrow.
Domain key rotation used to replay a settled payment	Mitigated by construction : K is immutable per domain; rotation = new domain, and cross-domain uniqueness rests on Assumption (I1) (§3).
Soundness: a false proof accepted	Mitigated by Groth16 soundness, <i>under</i> the circuit’s trusted setup (see below).
Attestation forgery: non-PSP submits a receipt	Mitigated : <code>ECDsa.recover</code> + registered-attestor check; EIP-712 domain binds <code>chainId</code> and the oracle address.
Front-running <code>attest</code> on a victim’s ν	Mitigated : a valid receipt requires the PSP signature over that exact struct.
Front-running <code>finalize</code> : an attacker copies a victim’s valid proof and receipt and submits <code>finalize</code> first, to steal the outcome	Not applicable : <code>finalize</code> is deliberately permissionless (any relayer may submit it, exactly like <code>attest</code>), and it is safe to be so because <code>SettlementV2</code> moves no asset and pays no address — it only transitions <i>the operation keyed by the proof’s own c_0</i> to <code>Finalized</code> and emits an event. Front-running changes <i>who paid the gas</i> , never <i>who benefits</i> : an attacker who submits it first still moves the same c_0 to <code>Finalized</code> , on behalf of the same operation, and gains nothing by being the caller. A downstream asset escrow built on top must preserve this: it must key its own release logic on the <i>stored</i> beneficiary from its own lock call, never on <code>msg.sender</code> of whichever transaction observes <code>isReleasable</code> , or this property would not transfer.
Payment is refunded after settlement (rejection of a false operation)	Mitigated within the dispute window , a deployment parameter (§2) in the form of a <code>cancel</code> deadline, that the

Trust decomposition. The claim-boundary box after the abstract states the ceiling; this states, claim by claim, what actually establishes each one.

Claim	Established by
Same amount, same <code>txid</code> across c_0/c	ZK (Property 3)
Tag deterministic in <code>e2eld</code> , single-use per domain	ZK + on-chain registry (Property 6)
E2EID genuinely bound to this settlement	ZK + PSP attestation (Property 4)
Counterparty commitment is the one that paid	PSP attestation + ZK cross-check (§5)
Settlement (credit availability) actually happened	PSP attestation alone (Assumption (I1))
Central-bank / rail truth	not established (§8, paths i–iv)
Liveness if the PSP never attests	protocol timeout (<code>cancel</code> , §4)
Finality against a later refund	deployment policy choice, not cryptography (§4.2)
Compliance-predicate policy (limit, sanctions/KYC roots)	policy-root authority (§5)

Only the first two rows are unconditional under (A1)–(A4). Every row below them routes through a named institutional or governance party, not through cryptography alone — which is the point of writing them out rather than letting “ZK-verified” read as covering the whole list.

Secret-compromise impact. Distinct secrets protect distinct things; conflating them overstates what any one compromise costs, in either direction.

Compromised secret	Impact
PSP attester signing key	Forge attestations within the receipt’s deadline , for any c_0/pc the attacker chooses; revocable for future attestations (§4).
Domain key k	Deanonymize every ν ever accepted in the domain back to its <code>e2eld</code> (the holder of k already has this by design — see “ ν is a selective-linkability primitive” above); does <i>not</i> by itself forge an attestation or a proof.
$salt_0$ (or $salt_p$) for one operation	Degrades that operation’s c_0 (or pc) hiding to the entropy of (amount, <code>txid</code>) (or the CPF/CNPJ space), both realistically enumerable (§4). Scoped to the leaked operation, not the domain.
salt (settlement-phase blinding)	Same degradation, for c instead of c_0 ; also operation-scoped.
Groth16 toxic waste (trusted setup)	Forge <i>proofs</i> for false statements, defeating soundness (Properties 2, 3, 4 and 6) entirely — the most severe compromise on this list, and the reason §8 states the reference setup’s development grade explicitly rather than in a footnote.
Registry owner key (policy admin)	Add a hostile attester or rewrite policy roots; mitigated only by a production multisig-with-timelock, not by this prototype’s single key.

Assumption (Trusted PSP (I1)). *The registered attester(s) sign `settled=true` only for instant payments the PSP observed as settled and credited, and hold an authoritative mapping ($txid \leftrightarrow e2eld \leftrightarrow amount \leftrightarrow debtor$) for each charge — i.e. the PSP’s own charge-and-payment records, not an assertion the institution hands it. Without this, nothing stops the PSP from correctly observing a settled payment and signing a receipt that pairs it with the wrong `txid`. It is what makes the*

recomputation of §4 (“the PSP evaluates the Poseidon itself”) mean anything: a PSP that recomputes c_0 over a self-consistent but wrong tuple produces a valid-looking receipt for the wrong charge. The construction reduces trust from “trust whoever submits the transaction” to “trust a specific PSP key, whose receipts expire and whose every attestation is auditable on-chain”, but does not eliminate it.

Reading the assumption charitably — and uncharitably. It is worth naming who the attestor is. The party in a position to confirm that the money was credited is the *receiver’s* PSP, and a false `settled=true` from it harms its own client, who would surrender the asset leg against a payment that never arrived. The incentive is therefore not aligned with the obvious attack, and the realistic residual risk is closer to “compromised PSP” than to “malicious PSP”. That is an argument for the design, not a proof: an institution under pressure, or an attacker holding the key within the receipt deadline, still produces a valid attestation.

Assumption (Trusted setup (S1)). *Groth16 soundness holds provided the per-circuit setup was performed such that the toxic waste was discarded. The reference artifacts use a development-grade setup; a production deployment requires a multi-party ceremony. This is a known limitation, not a claim.*

Path to removing the PSP assumption. None of the four paths below is part of the evaluated construction of §7–§9; each is discussed to make legible the ceiling on the trust reduction, not to claim it as already delivered. The endgame is a *trustless* attestation, and it is worth being precise about what each candidate actually removes. (i) *A signature from the SPI/Bacen infrastructure* over (txid, e2eld, settled) is the only option that takes the PSP out of the trusted computing base entirely. As a *synchronous* service — an online signing endpoint on the critical path of every settlement — it is not publicly available today, and asking for one is a heavy institutional request. The asynchronous variant is a far lighter ask and deserves its own entry; we name it as path (iv) below. (ii) *zkTLS/DECO* [7] proves what the PSP endpoint *responded* over a TLS session. This is a real gain — it removes trust in the attestor signing key and in the reporting channel — but it does not make the PSP honest about the status: a PSP that reports a false `settled` over TLS yields a valid zkTLS proof of a false statement. Presenting zkTLS as “removing the trusted PSP” would overstate it. (iii) *Dual attestation via open banking* is a worthwhile intermediate step that we register here as future work. Under Open Finance Brasil, responses from the *payer’s* institution are signed (JWS, PS256) and carry the `endToEndId` and status. Combining that with the receiver’s PSP attestation turns “trust a PSP” into “trust that two regulated institutions on opposite sides of the same E2EID are not colluding” — still an assumption, but a materially stronger one. (iv) *Optimistic attestation with retrospective rail anchoring.* The attestation does not have to be trustless *at settlement time* to stop being merely trusted: it is enough that it becomes verifiable *after the fact*. The receiver-PSP receipt of §4 gives optimistic finality — the operation reaches FINALIZED, and IRREVOCABLE after the dispute window, exactly as today — and a rail-level statement, landing minutes or days later, either hardens it or contradicts it. Concretely, the state machine gains one orthogonal flag, `RailAnchored`, settable at any time after attestation: an anchor consistent with the receipt upgrades the operation’s evidence from “a regulated party said so” to “the rail said so”, and, *under a stated completeness guarantee for the signed root* (below), a receipt that cannot be anchored stands contradicted: once the rail statement for that period exists, either the institution presents the membership proof — hardening — or its absence after a grace period becomes the signal, and any party holding `e2eld` and k (the attesting PSP, or an auditor under the disclosure of §5) can prove non-settlement outright with the same SMT non-membership predicate the sanctions circuit already uses. **This last step is only as strong as the completeness of the signed**

root: absence from a Merkle root proves non-settlement only if the root is published with an explicit claim of the form “this root commits to the complete set of E2EIDs settled in interval T ”, made by the same central-bank infrastructure that signs it — otherwise a delayed batch, an incomplete window, a synchronization gap or a publisher omission is indistinguishable from genuine non-settlement, and absence proves nothing. This is a real, separate assumption on top of the signature itself, and path (b) below should not be read as delivering non-membership proofs until that completeness claim exists for the specific rail artifact used. At that point Assumption (I1) stops meaning *trusted* PSP and starts meaning *accountable* PSP — the attester can still lie, but the lie becomes provable and attributable ex post, which is the deterrent regulated institutions actually answer to. Two realizations, in increasing order of independence. (a) *Reuse what the rail already signs:* the SPI’s own settlement messages — the `pacs.002` status report delivered to the payer’s PSP and the settled `pacs.008` forwarded to the receiver’s PSP, both cataloged ISO 20022 messages of the SPI, specified in Volume VI (Instant Payments) of the SFN Service Catalog and carried over the RSN [14] — are statements of precisely the fact at issue, held today by the PSPs of both legs. We do not claim a specific signature algorithm or certificate chain here: the SFN Service Catalog documents the message set and transport, but we have not independently confirmed the exact signature scheme against the Bacen security manual, and state only what the catalog establishes. Verifying that signature inside a circuit (the zk-email line, with the rail’s key as public anchor) or inside an attested TEE, and exposing only the tag, turns an artifact that already exists into a public anchor; the natural prover is the receiving institution itself, which already holds `e2eld` and k , so no new custodian of the linkage secret is created. (b) *The policy ask, restated cheaply:* rather than an online signing service, a periodically *signed accumulator* of settled E2EIDs — a Merkle root, exactly the SMT machinery of §5 — published by the central-bank infrastructure would suffice: the institution proves membership of its `e2eld` under the signed root in zero knowledge, exposing only ν . Asynchrony is what makes (b) plausible: it puts nothing on the settlement’s critical path, batches trivially, and tolerates delay by construction, because the optimistic leg already closed the operation. Two boundaries, stated so this path is not oversold. The rail anchors *settlement at the SPI*, not *credit availability*, so the receiver PSP’s receipt remains necessary for the settled-means-credit semantics of §4.2 — although returns (`pacs.004`) traverse the SPI symmetrically and can be anchored the same way, which would also make the refund window of §4.2 visible on-chain ex post rather than invisible. And the lesson of §4 applies unchanged: neither the signed rail message nor any statement carrying the E2EID in the clear ever goes on-chain; verification happens off-chain or in-circuit, and only the tag and a boolean are anchored.

Path	Removes	Does not remove
This work’s PSP receipt	submitter trust (anyone can relay)	PSP honesty about settled
(ii) zk-TLS/DECO	attestor signing key, reporting channel	PSP correctness of the reported status itself
(iii) Dual PSP (open banking)	single-institution trust	collusion between the two
(i) Central-bank signature	PSP trust entirely	nothing (assuming SPI/Bacen key custody) — but not a synchronous service today
(iv-a) Signed rail artifact (pacs.002/004)	PSP trust, retrospectively	freshness (evidence lands after finality)
(iv-b) Signed accumulator + membership	online signing service (asynchronous)	completeness of the root (stated above)

None of these rows composes with another automatically: (iv-a) and (iv-b) still rest on the SPI/Bacen infrastructure choosing to publish the underlying artifact, which is a policy fact about the rail, not a cryptographic one.

Privacy model: what is and is not hidden. The construction hides, on-chain, the payment *amount*, the *parties*, and the payment identifiers (`e2eld`, `txid`): these values appear only through Poseidon commitments or ZK witnesses, and are not carried in plaintext by the attestation receipt, with negligible leakage under (A2). It does *not* hide: (i) the *existence* and *timing* of an operation — a settlement transaction is public; (ii) the *on-chain addresses* that submit the settlement and relay the attestation, and the *attestor address* itself — address-level linkability is out of scope and, if required, must be handled by an external anonymity layer; and (iii) the fiat leg itself, which the settlement rail (the SPI and the PSPs) observes by construction — Harpo conceals the amount and identifiers *on-chain*, not the fact that a payment traversed the rail. The regulator’s confidential view is governed on-chain (budget, threshold, log; §5, [8]).

Confidentiality is not unlinkability, and the construction does not claim the latter. Even an observer who never learns amount, `e2eld` or party can still correlate operations by: the same `pc` recurring across settlements (a counterparty that transacts more than once is linkable, by design — `pc` exists precisely so predicates can be evaluated consistently against the same identity); the same submitting or relaying address; the same attestor address (which operation went through which PSP); the same domain anchor K ; and ordinary transaction timing. None of these is mitigated by this construction; an external anonymity layer would be needed for any of them, and none is currently in scope.

Property	Protected?
Amount confidentiality	Yes
<code>e2eld</code> / <code>txid</code> confidentiality	Yes
Party identifier confidentiality	Yes
Transaction existence and timing	No
Submitting / relaying address privacy	No
Attestor (PSP) address privacy	No
Cross-operation linkability (repeated <code>pc</code> , K , addresses)	No

9 Implementation and Benchmarks

The circuit is implemented in circom 2.2.3; contracts in Solidity 0.8.27; the Groth16 verifier is snarkjs-generated (snarkjs 0.7.6). All figures below are *measured*, not estimated, on an 11th-gen Intel Core i5-1135G7 (mobile, containerized).

9.1 Circuit

Constraint counts depend on the circom optimization level, and reporting them without it is ambiguous; we report both `-O1` (non-linear plus linear, circom’s default) and `-O2` (linear constraints eliminated), with `-O2` as the primary figure, for the primary settlement circuit and for the retained unkeyed ablation circuit discussed below.

Circuit	-O2	-O1 (n.l. + lin.)	Public / private
<code>settlement_verify_v2</code> (primary)	1,251	1,266 + 1,524	5 / 8
<code>pix_nullifier_verify</code> (keyed, v1)	537	543 + 710	2 / 5
<code>pix_payment_verify</code> (unkeyed, ablation)	510	516 + 635	2 / 4

The unkeyed circuit `pix_payment_verify` is retained only as an *ablation*: it is the variant §3 declares vulnerable (RT-5, deanonymization via the unkeyed tag), kept measured here specifically so its cost can be compared against the keyed `pix_nullifier_verify` and against the primary `settlement_verify_v2` — these are three distinct circuits, and none of the figures above should be read as three measurements of the same one. The primary circuit has 1,260 wires and 4,229 labels, five Poseidon gadgets (arities 3, 4, 2, 1, 2), and Groth16 proofs of 8 field elements (≈ 256 bytes) over BN254.

9.2 Performance

Prover and off-chain verifier measured in-process with the snarkjs WASM backend (driving snarkjs through its CLI instead adds ≈ 390 ms of Node.js start-up per run, which we avoid by measuring in-process). The figures in the table below are *single warm runs* on the stated hardware, not a median over repeated trials: at these circuit sizes (≈ 10 – 15 ms of verification, ≈ 100 – 150 ms of proving) run-to-run noise is comparable to the differences between circuits — the ablation circuit (510 constraints) timing *above* that of the 537-constraint circuit cannot be distinguished from noise without a dispersion figure, and we do not have one on the stated hardware. We did run a dispersion measurement (median $\pm$ SD over $N = 30$ warm-up-then-timed runs, via `scripts/bench-prove.cjs`) on a separate, uncontrolled machine: prove $594.8 \text{ ms} \pm 86.6$, verify $31.0 \text{ ms} \pm 10.6$. Both figures are several times the single-run numbers below, which is consistent with an uncontrolled machine rather than with the in-process/CLI distinction, so we do not substitute them into the table; a same-hardware ($N \geq 30$, median $\pm$ SD) re-run on the stated i5-1135G7 is listed as future work (§11) rather than reported here as if already performed.

Circuit	Prove	Verify (off-chain)	verifyProof gas
<code>settlement_verify_v2</code>	151.2 ms	13.6 ms	243,194
<code>pix_nullifier_verify</code>	110.5 ms	14.0 ms	—
<code>pix_payment_verify</code> (ablation)	130.1 ms	14.7 ms	238,387

We deliberately do *not* quote a measured delta against the two-signal ablation. The two verifiers (238,387 gas for `pix_payment_verify`, 243,194 for `settlement_verify_v2`) were generated and

measured under conditions we cannot certify identical — distinct `snarkjs` invocations, potentially distinct trusted-setup runs — so the two figures are not a controlled A/B on signal count alone. As a sanity check: the naïve difference between them (4,807 gas) is *smaller* than the cost three extra public signals must add on their own under EIP-1108 (one `alt_bn128` scalar multiplication plus one addition per signal, $\approx 6,150$ gas each, $\approx 18,500$ for three), which confirms the two measurements are not directly comparable rather than telling us what the true per-signal delta is; we do not report a delta figure, measured or expected, until a same-template re-measurement (§11) produces one we would stand behind. The dominant cost of either verifier remains the four-pairing check ($\approx 181,000$ gas: $45,000 + 34,000 \times 4$, EIP-197).

Gas of the complete calls. Reporting only the isolated verifier understates what a deployment pays. The figures below are entire transactions on the XDC Apothem public testnet (chainId 51), taken from transaction receipts and listed with their hashes in the artifact manifest.

Call	Gas
<code>lock(c0, deadline)</code> — escrow locks on the agreed terms, with a liveness deadline (§4)	74,982
<code>attest(receipt, signature)</code> — ECDSA recover, checks, tag storage (v2: 88,646)	90,913
<code>finalize(proof)</code> — complete settlement (v2: 352,983)	353,315
of which <code>verifyProof</code> (Groth16, 5 public signals)	243,194
<code>cancel(c0)</code> — after the deadline, Locked $\rightarrow$ Expired	30,259
<code>attestReversal(reversal, signature)</code>	63,529

Figures are the current reference deployment (with `pc` in the receipt, the `finalize` cross-check, and the `lock/cancel` liveness timeout); v2 figures are given alongside for the two calls whose struct changed relative to it. `attest` costs 2,267 gas more than v2 (one extra `bytes32` field in calldata and in the EIP-712 digest); `finalize` costs 332 gas more (one `keccak256` over 64 bytes plus the extra calldata word for the cross-check) — both consistent with the single-word binding storage of §4 rather than a fresh storage slot. The complete settlement transaction costs 353,315 gas. An *isolated* call to the verifier contract with the same proof and public signals costs 243,194 gas — including its own 21,000 base transaction cost and its own proof calldata, neither of which recurs when the verifier is instead called *internally* from `finalize`, so 243,194 is an upper bound on the verifier’s share of the 353,315, not a component that subtracts cleanly to give the remainder. With that caveat, the bulk of the complete transaction’s cost is still the Groth16 verification; the rest is the oracle cross-check (the receipt must be attested *and* carry the same (`c0`, `pc`) the escrow and the proof agree on), the domain-anchor check, the tag consumption and the state transition. Quoting the isolated verifier figure alone, unqualified, would both overstate its call-time cost and understate the complete transaction’s real cost — which is why we report the complete-call figure as primary and give the isolated figure only with this caveat attached. `cancel` is cheap by comparison — a single state write and no cryptographic check — which is expected: the liveness path exists precisely so a stuck operation costs little to release.

9.3 ZK-Circuit Constraint Analysis

The settlement and compliance circuits were checked with Ecne [11], which verifies that an R1CS constraint system is *well-determined* — that every signal is uniquely fixed by the inputs, i.e. no *under-constrained* signal, the dominant ZK-circuit bug class. We call this section *constraint analysis* rather than *formal verification* deliberately: what follows establishes the absence of one bug class in the R1CS, not functional correctness, not protocol security, and not soundness of the underlying cryptographic construction (see Scope of the claim, below).

A pitfall worth naming: the vacuous verdict. Running Ecne against our circuits initially printed “*has sound constraints (no trusted functions needed)*” for every one of them. Taken at face value that reads as a genuine result. It is not: Ecne decides its verdict by comparing how many *target* variables it uniquely determined against the number of targets, and the targets are the *public outputs* of the R1CS. Our circuits were written in the style `signal input c; ...; c === h.out;` — the public values are declared as inputs and pinned by an equality constraint. Such a circuit compiles to an R1CS with `nPubOut = 0`, so the target set is empty and the verdict reduces to $0 = 0$: the tool prints “sound” without having demonstrated anything about the circuit. We caught this only by reading the Ecne source (the criterion `target_unique == length(target_variables)`) against the R1CS headers of our own circuits, every one of which has zero public outputs; we report it here because a reader who trusted the printed verdict, as we initially did, would be wrong to. The observation generalizes beyond this construction, and we state it as a recommendation: any published Ecne verdict for circuits written in the input-plus-equality-constraint style — a common circom idiom — should be re-read against `nPubOut` in the R1CS header, because the verdict is vacuous whenever it is zero, and the tool does not say so. Reports should quote targets-solved / targets alongside the verdict, as the tables here do; a verdict without its target count is not evidence. We intend to report this behavior upstream at github.com/franklynwang/EcneProject.

Verification under a corrected methodology. Two changes make the analysis meaningful: declare the derived public signals as `signal output` so the target set is non-empty, and run the analysis on the *unoptimized* (`-00`) constraint system, since the `-02` optimizer folds the linear structure that Ecne’s symbolic solver relies on. The circuit is otherwise identical — same source, same 1,251 constraints at `-02`. Under this methodology we obtain genuine, non-vacuous results:

Circuit	Outputs	Targets	Signals	Verdict
<code>settlement_verify_v2 (-00)</code>	declared	5/5	4,229/4,229	sound, no trusted fns
<code>kyc_inclusion_verify (-00)</code>	declared	1/1	18,230/18,248	sound, no trusted fns
<code>settlement_verify_v2 (-02)</code>	declared	4/5	1,016/1,260	<i>inconclusive</i>

The KYC circuit’s target (its single declared output) is solved (1/1), so none of the 18 signals Ecne leaves undetermined is a target — that much follows directly from the target count and is not a guess. What those 18 signals *are* is not yet identified against the `.sym` file; we do not report an unverified guess about their identity here and list identifying them as future work (§11) instead. The `-02` row shows the tool failing to conclude on an optimized system: “inconclusive” means Ecne’s solver could not determine one target, not that the circuit is unsound. This is itself an instance of the discipline the pitfall above demands: reporting the target count (4/5, not simply “fail”) is what lets a reader see that the analysis is incomplete rather than negative.

Scope of the claim. Ecne proves well-determinedness. It does *not* prove functional conformance with a specification: a circuit can be perfectly well-determined and still compute the wrong predicate. The guarantee is the absence of one important bug class, not correctness. Tools in the Picus [12] line attack the same under-constrainedness question by a different technique and would be a valuable cross-check; we have not run it, and list it as future work. This is complemented by circomspect [13] (a static linter for circom) and a manual audit against a systematic ZK-vulnerability taxonomy. On the public-key side, the ElGamal/BabyJubJub circuits validate every input point (on-curve via `BabyCheck`, plus a small-order rejection $8 \cdot P \neq \mathcal{O}$) to preclude invalid-curve and small-subgroup confinement.

9.4 Reference deployment and reproducibility

What follows is deployment and implementation evidence, and we are explicit about which claim each kind of evidence in this paper actually supports, so that none is read as standing in for another: *cryptographic evidence* (Groth16 verifies, commitments bind — §7) is what backs the formal properties; *constraint-analysis evidence* (Ecne well-determinedness — §9.3) is what backs the absence of under-constrained signals; *implementation evidence* (the transactions below: correct calls succeed, incorrect ones revert) is what backs “the code does what §3–§4 describe”; and *deployment evidence* (a live public testnet) is what backs “this runs against a real EVM under real gas costs”, nothing more — a testnet deployment is not security evidence, and we do not present it as such. The construction described in this paper — the two-phase circuit, the attestation oracle and the composition that ties them together — is deployed and exercised on the XDC Apothem public testnet (chainId 51). The settlement path was run end to end: the escrow locks on c_0 before any payment exists, the PSP receipt over c_0 is attested, and the Groth16 proof closes the operation. Crucially, *no amount, E2EID or charge identifier appears in any of those transactions* — the confidentiality property central to this design, and a reader can check it directly against the calldata.

The acceptance rules were also exercised negatively, on-chain, in two batches. The first covers receipt validity: a receipt signed by a non-attestor, a receipt with `settled=false`, an expired receipt, and a replay of an already-attested tag were each mined and reverted. The second covers the value-consistency mechanism itself (§3) directly: a receipt whose c_0 diverges from the one the proof opens (*case 5a* — a PSP that recomputed c_0 over the wrong settled amount would produce exactly this), a receipt whose `partyCommitment` diverges from the one the proof opens (*case 5b* — without this check `partyCommitment` would be a decorative public signal, since a prover could attach any counterparty commitment it likes to a valid proof), and a proof generated under a domain key k' different from the registered domain-anchor K (*case 6* — the tag-uniqueness argument of §3 only holds if the proof is bound to the registered domain). A fourth case exercises the reversal path itself (*case 7*): a short-window instance (600s, deployed only to make this exercisable in minutes) is settled, then reversed within the window via `attestReversal`; `isReleasable` immediately reports `false`, and 660s later — past the window that would otherwise have made an unreversed operation irrevocable — `isIrrevocable` on that tag is still `false`. A reversal blocks release permanently, not merely until the window closes. A fifth case (*case 8*) exercises the liveness timeout added in this revision (§4): `cancel` on a still-Locked operation reverts before its `lock` deadline, succeeds once the deadline passes (moving it to `Expired`), and a genuinely valid proof submitted to `finalize` afterward reverts, on-chain — `cancel` and `finalize` are exercised as the mutually exclusive pair §4 states them to be, not just asserted to be so. All nine cases were submitted with explicit gas so that they would be recorded on the chain rather than stopped by client-side simulation, leaving a hash a reader can look up.

Contract addresses, every transaction hash, the public signals of the settled operation, and the exact commands that reproduce each number in this section are listed in `docs/eprint/ARTIFACTS.md` of the reference repository, <https://github.com/harpo-zk/proof-of-instant-settlement>, at commit `d38feb77919c89c3697d7bc59c9d88aba0746a68`. Reproducibility claims in this section are claims about that exact commit.

A reading note for those transactions: in the reference deployment the deployer, the registered attestor and the registry owner are the same key (`0x7c20...a49`) — a prototype convenience, not a design property. The negative cases must be read accordingly: the non-attestor case is signed by a second key, `0x19E7...ff2A`, and a production deployment separates the three roles (multisig owner, §8).

10 Related Work

Proving fiat payment to release on-chain value. Proving that a fiat payment happened in order to release on-chain value is not new. ZKP2P [5] has done exactly that in production since 2023, using zk-email and zkTLS over consumer payment rails, and is the honest prior art for the general mechanism; it does not target the institutional setting and has no confidential-compliance model. Canton [6] is the incumbent for institutional delivery-versus-payment with privacy, on a permissioned substrate; Brazil’s Drex pilots occupy the same regulated space with a central-bank-operated ledger. Our position is the intersection those leave open: a public EVM chain, an institutional (PSP-attested) rail binding, and compliance answered as predicates.

Confidential transfer on EVM. Rayls/Enygma [3] (Parfin) is the closest peer in the confidential-transfer family: same regulator context, same Groth16/Poseidon/BabyJubJub primitives, central-bank pilots, and it provides confidential, EVM-compatible institutional settlement and delivery-versus-payment primitives in its own right, with a *global* auditor view key — we do not attempt a detailed architectural comparison beyond that one point, since it is what our own auditor model (predicate/scoped, §5, §6) is contrasted against in the table below, and nothing more is needed for our claim. Zeto [4] and Nightfall provide generic confidential transfers on EVM. To the best of our knowledge, none of these binds a real instant-payment rail in the sense evaluated here: they answer “was the on-chain transfer well-formed”, not “did the fiat settle”. Our contribution is orthogonal: binding an external instant-payment rail event to confidential EVM state, composable with what these systems already do downstream of that point (§10’s closing thesis makes this precise). For post-quantum auditability, Qurrency [10] adds ML-KEM-based auditor access to Zeto on a Groth16/circom stack; we reuse its open-source ML-KEM circom implementation (§6) and bind it to our rail-anchored commitment. Our compliance predicates (§5) differ from Zeto’s KYC/audit variants in the disclosure model: where those let an authority decrypt the value, ours disclose only a boolean.

	ZKP2P	Rayls	Zeto	Qurrency	This work
Fiat-rail binding	yes	no	no	no	yes
Institutional / regulated rail	no	yes	n/a	n/a	yes
Public EVM chain	yes	partly	yes	yes	yes
Confidential transfer	no	yes	yes	yes	yes
Auditor model	n/a	global key	decrypt	decrypt (PQ)	predicate / scoped
Compliance w/o disclosure	no	no	no	no	yes
Post-quantum audit channel	no	no	no	yes	preliminary
Production maturity	yes	pilots	yes	yes	prototype

Table: feature comparison. “decrypt” means the auditor recovers the plaintext; “predicate” means the auditor learns only a yes/no. On *maturity* the comparison inverts: ZKP2P is in production and Zeto/Qurrency are backed by LF Decentralized Trust; this is a single-maintainer prototype. The edge claimed here is the design and the rail binding, not deployment maturity.

Stated as a thesis, not a feature list. The table above is useful for placement, but it understates the actual claim, which is architectural rather than feature-by-feature: *the novelty here is not confidential transfer itself — Zeto and Rayls/Enygma already do that well, and Paladin’s roadmap for atomic composition between private on-chain components is a serious, active answer to that problem on its own — it is the cryptographic binding layer between an external, real-world instant-payment event and confidential EVM state.* Zeto and Paladin answer “is this confidential

on-chain state well-formed”; this construction answers “does a specific real payment, attested by a regulated PSP, correspond to this confidential on-chain state”, and hands the resulting commitment forward, unopened, to whatever confidential-asset system consumes it. Read this way, the natural relationship to Zeto/Paladin is not competition but composition: this construction is upstream of them, converting an external fiat event into a confidential commitment they could consume as an input, not a replacement for what they do downstream of that point. We have not built or measured that composition (§11); the claim here is architectural positioning, not a demonstrated integration.

11 Open Problems and Future Work

1. **Trustless attestation** (§8): a synchronous central-bank signature (the heavy ask); optimistic attestation with retrospective rail anchoring (path iv) — including its cheap policy form, a periodically signed accumulator of settled E2EIDs, and its engineering form, in-circuit or in-TEE verification of the rail confirmations the PSPs already hold (`pacs.002/pacs.004`); dual attestation via open banking; or zkTLS over the PSP endpoint — each removing, or converting into ex-post accountability, a different part of the trust. Of these, the retrospective anchor is the only one deployable in principle against artifacts the rail signs today, and it composes with the state machine of §4.2 as an orthogonal flag rather than a change to the settlement path.
2. **Dispute path beyond timeout**: `cancel` (§4) closes the liveness gap with a deadline-based escape, which is sufficient but coarse — it does not let a counterparty contest a still-live operation before the deadline, or distinguish “PSP is slow” from “PSP is refusing”. A richer off-chain dispute path (e.g. an appeals window an aggrieved party can trigger before the deadline) is future work on top of the timeout primitive.
3. **Attestor privacy**: hiding *which* PSP attested, via a ring or threshold signature over the registered attestor set.
4. **Atomic DvP, and a real composition with a confidential-asset system**: the two-phase commitment makes the asset leg lockable on c_0 ; composing it with the payment leg into a single atomic transition remains to be built and measured, and the strongest version of this is an actual integration — even at testnet scale — with a system like Zeto or Paladin consuming this construction’s commitment as an input to a private asset transfer, turning the architectural positioning of §10 into a demonstrated one.
5. **Production trusted setup**: a real multi-party ceremony.
6. **Primitive agility**: migration to Poseidon2 and to a universal-setup proving system (PLONK/Halo2).
7. **Full post-quantum binding** (§6): the ceremony for the 1.62M-constraint ML-KEM binding circuit on production hardware.
8. **Verification depth** (§9.3): cross-check with Picus [12], extend the analysis to the optimized constraint system, identify the 18 undetermined non-target signals of `kyc_inclusion_verify` against its `.sym` file, and move from well-determinedness towards functional conformance.
9. **Same-environment benchmark re-run** (§9): a dispersion measurement (median $\pm$ SD, $N \geq 30$) of proving/verification time on the stated i5-1135G7, and a same-template gas re-measurement of the two-signal and five-signal verifiers generated from the same snarkjs codegen, to report a per-signal delta we would stand behind.

10. **The attested counterparty commitment as a standalone primitive:** a regulated institution attesting $pc = \text{Poseidon}(\text{id}, \text{salt}_p)$ over the identifier it processed is a reusable identity anchor — third-party ZK predicates (credit, licensing, jurisdiction) can consume the same pc without any change to this protocol. Generalizing it beyond the three predicates shipped here is future work.

Availability and Honesty Statement

The settlement circuit, its snarkjs-generated Groth16 verifier, the attestation-oracle contract, the compliance predicates, and the end-to-end flow scripts are available in the reference repository, <https://github.com/harpo-zk/proof-of-instant-settlement>, at commit d38feb77919c89c3697d7bc59c9d88aba0746 together with the artifact manifest described in §9. This is a research prototype validated on a public testnet; it has no production deployment, and the reference implementation is maintained by a single maintainer. Claims are limited to what the listed transactions demonstrate, and where a tool’s raw output would naturally be read as stronger evidence than it is — notably the Ecne verdicts of §9.3 — we say so explicitly rather than let the stronger-sounding reading stand.

References

- [1] J. Groth. *On the Size of Pairing-based Non-interactive Arguments*. EUROCRYPT 2016.
- [2] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. USENIX Security 2021.
- [3] M. Yaksetig, J. Xu. *Rayls: A Novel Design for CBDCs*. IACR ePrint 2025/1639 (Parfin). See also M. Yaksetig, P.M.F. Pereira, S. Yang, M. Nejadgholi, J. Xu. *Rayls II: Fast, Private, and Compliant CBDCs*. IACR ePrint 2025/1638 (the Enigma protocol, revised).
- [4] LF Decentralized Trust / Paladin. *Zeto: UTXO-based confidential tokens with ZK proofs*.
- [5] ZKP2P. *Permissionless fiat-to-crypto settlement from zero-knowledge proofs of payment (zk-email / zkTLS)*. zkip2p.xyz.
- [6] Digital Asset. *Canton Network: a privacy-enabled network of interoperable applications for institutional settlement*.
- [7] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, A. Juels. *DECO: Liberating Web Data Using Decentralized Oracles for TLS*. ACM CCS 2020.
- [8] Harpo. *Selective Disclosure: scoped, threshold, logged audit*. [docs/SELECTIVE-DISCLOSURE.md](#), this repository.
- [9] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. 2024.
- [10] *Qurrency: a quantum-secure, private, and auditable platform for digital assets*. IACR ePrint 2026/015. Implementation: LF Decentralized Trust / Paladin, [LFDT-Paladin/zeto](#).
- [11] F. Wang et al. *Ecne: Automated Verification of ZK Circuits* (R1CS soundness / well-definedness). github.com/franklynwang/EcneProject.

- [12] S. Pailoor et al. *Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs*. PLDI 2023.
- [13] Trail of Bits. *Circomspect: a static analyzer for Circom*. github.com/trailofbits/circomspect.
- [14] Banco Central do Brasil. *Catálogo de Serviços do SFN, Volume VI — Pagamentos Instantâneos*, versão 5.12. https://www.bcb.gov.br/content/estabilidade/financeira/cedsf/Catalogos/Catalogo_de_Servicos_do_SFN_Volume_VI_Versao_512.pdf.
- [15] Receita Federal do Brasil. *Instrução Normativa RFB nº 2.229, de 15 de outubro de 2024* (CNPJ alfanumérico, implantação progressiva a partir de 31/07/2026). <https://www.gov.br/receitafederal/pt-br/aceso-a-informacao/acoes-e-programas/programas-e-atividades/cnpj-alfanumerico>.

A Pix instantiation of CanonicalID

This appendix gives $\text{CanonicalID}_{\text{Pix}}$, the concrete canonicalization rule this prototype and its pilot deployment use for the Pix rail (§3). It is a deployment profile, not part of the core protocol: any other instant-payment rail plugs in its own $\text{CanonicalID}_{\text{rail}}$ in its place, unchanged elsewhere.

CPF/CNPJ. The bare ASCII characters of the identifier with all punctuation stripped and no padding — 11 decimal digits for a CPF; 14 characters for a CNPJ, upper-cased, exactly as extracted from the debtor field of the payment message. Since Instrução Normativa RFB nº 2.229/2024 [15], rolled out from July 31, 2026, the first 12 of those 14 characters may be digits or uppercase letters (existing all-numeric CNPJs remain valid unchanged); the final two check digits are computed by the Receita Federal’s modulo-11 algorithm over the ASCII value of each character and are always numeric — so $\text{CanonicalID}_{\text{Pix}}(\text{CNPJ})$ needs no branching for the two formats, only the fixed 14-character, upper-cased, punctuation-stripped rule above.

E2EID / txid. The exact bytes returned by the PSP API: UTF-8, no case-folding, no trimming, no re-serialization (§2). This is the same rule stated in §2 for these two fields; it is repeated here only for the appendix to be self-contained as the Pix deployment profile.